# PID Controllers and Matlab / Simulink

## 1. Aims and Objectives

The aim of this task is to develop a simple model showing basic PID control of a DC Machine in Matlab / Simulink, while the objectives are a better understanding of how machines can be controlled, Matlab /Simulink, and the 'Simscape: Specialized Power Systems' Library. The general principles used in developing these models can be applied to any future simulation work including those looking at combined power electronic converter and machine systems. This handout aims to give you an understanding of the performance and interaction of each.

## 2. Developing a Simple PID and Machine Model using Simulink.

The following instructions assume you are using Matlab 2021a or later. Within Matlab open the Simulink library browser – this will give you a long list of libraries, each of which contains a large number of blocks which can be used to develop any given model. There are two options to open Simulink as shown in Fig. 1, either click on the button or type ***'Simulink'*** into the command window, this will open the ***'Simulink Start Page'*** (Fig. 2). Choose ***'Blank Model'***. A blank model called ***'untitled'*** should appear; save this is a suitable place. To obtain the libraries, click the ***'Library Browser'*** button as shown in Fig. 3, the library as shown in Fig. 4, should appear.

For this model we are using blocks from the standard ***'Simulink'*** library and the ***'Simscape: Specialized Power Systems'*** library – this contains most of the basic elements required for simulating power electronics and electric drive systems. Feel free to explore the full contents of this library in your own time. Here we will just focus on the small number of components required to construct the PID and machine model.

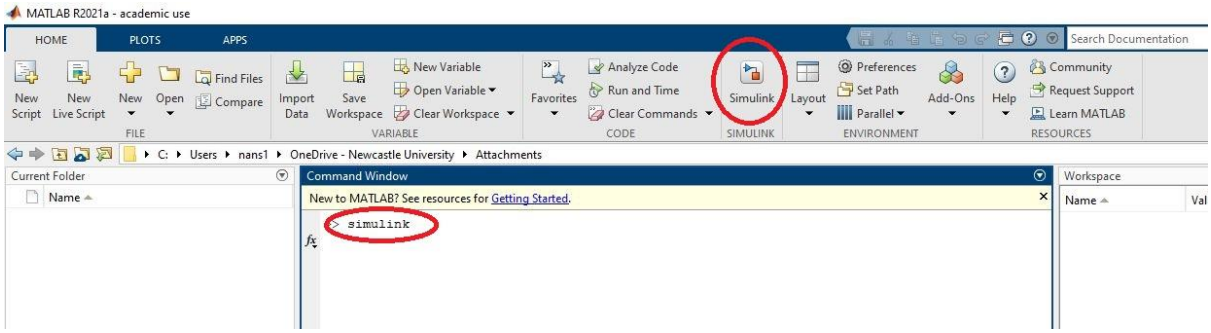


**Figure 1: Open Simulink Options**

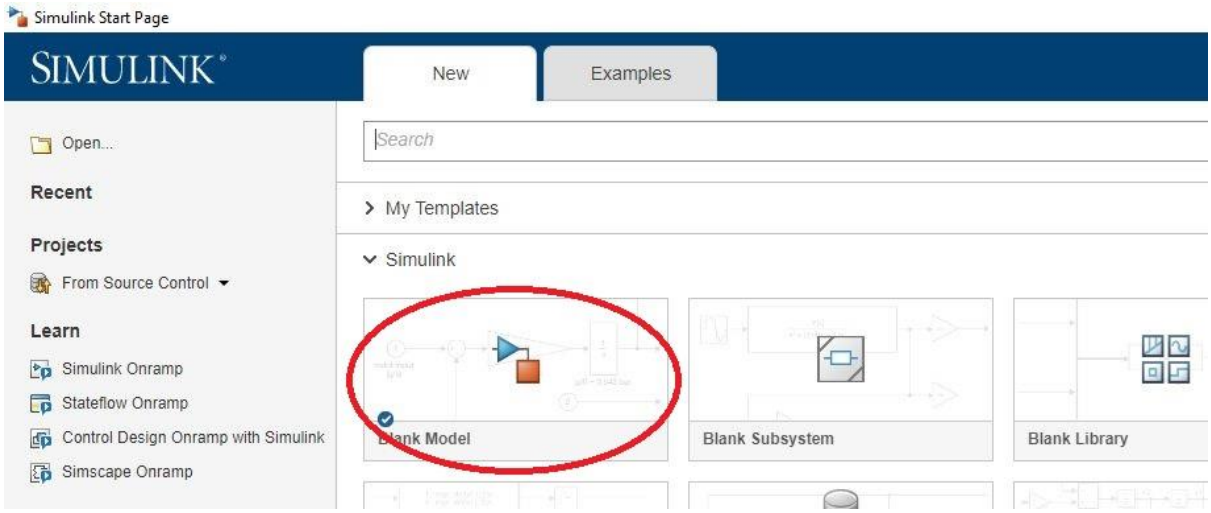


**Figure 2: Create a blank model**

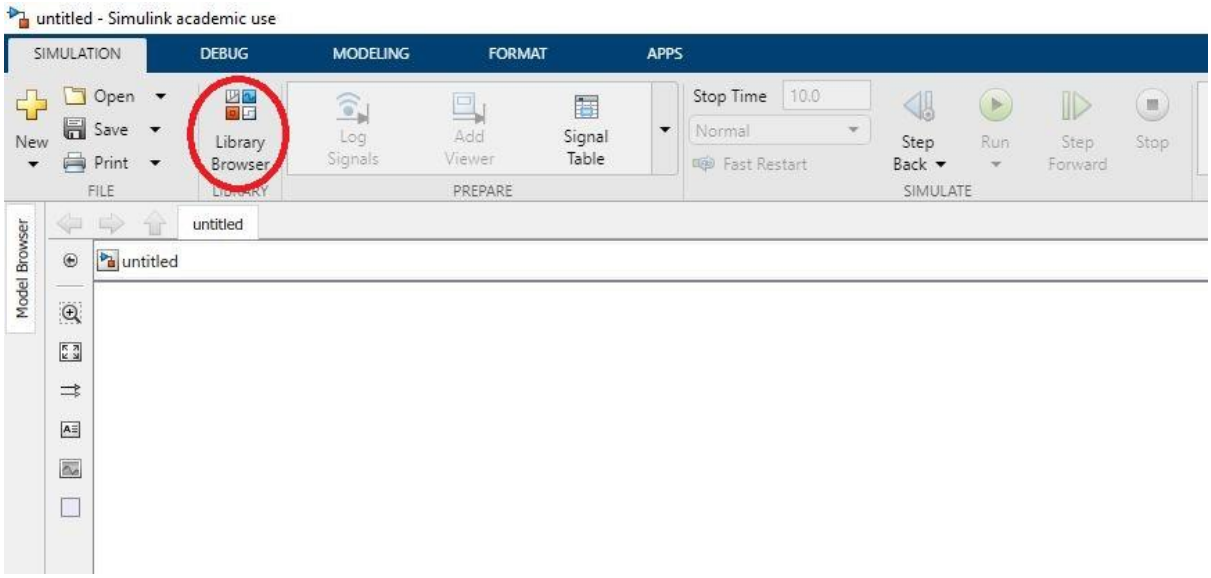


**Figure 3: To open Simulink Library**

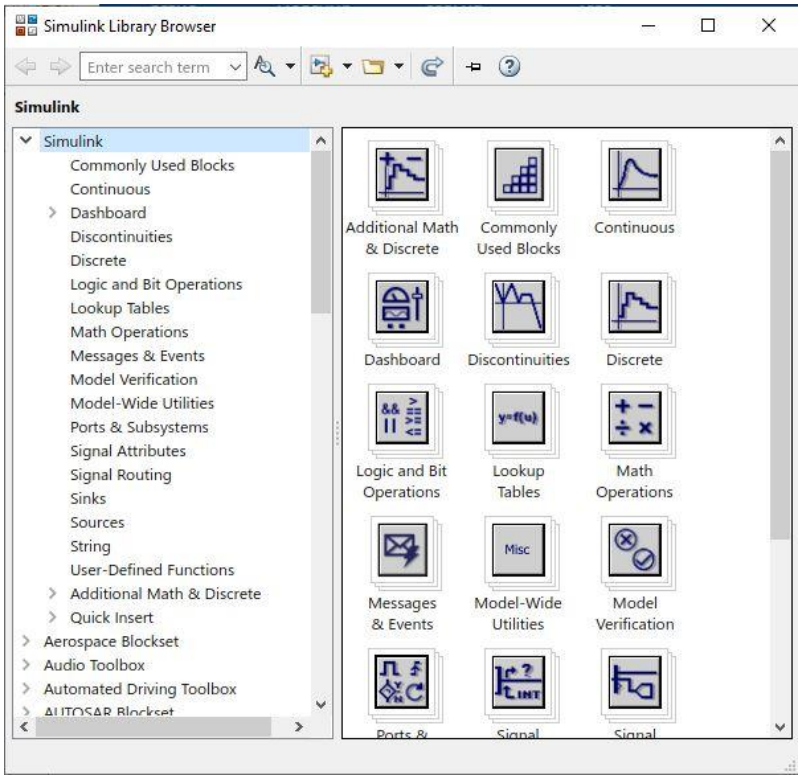


**Figure 4: Simulink Library Browser**

## 2.1 Some General Tips on Simulation Modelling

- If it works first time, think yourself lucky. It is rare that a simulation model will work as expected first time round – like it or not, you are going to have to debug the simulation!
- When debugging a simulation, consider the problem logically and where possible simplify things as much as possible. Try to confirm that the subsystems within the model behave properly first.
- No matter how small or obvious you think it is, never assume something is okay until you have checked it properly. Has this been verified on the scope?
- Most important of all: A simulation model should generally be used to confirm your understanding of the system. It should not be used to generate answers to something you do not understand!!

# 3. Introduction to the DC Machine

Hopefully this is just a refresher for you after the information you learnt in ENG1004.

In Brushed DC motors, a commutator and brushes are used to convert a DC current in the armature coil to an AC current. However, the arcing produced by the armature coils on the surfaces of the brush and commutator generate heat, wear, and electromagnetic interference (EMI), and these are the major drawbacks of this machine.

In Brushed DC motors, a voltage $v_a$ is applied to the armature circuit which has a resistance $R_a$ and an inductance $L_a$. This voltage will cause a current $i_a$ to flow and an armature magnetic field to be created. A rotor magnetic field is also created using either permanent magnets or a separate field coil, and the interaction between the two magnetic fields results in an applied torque to the motor armature that results in a rotation of the motor.

The torque generated by the motor is proportional to the armature current and the strength of the rotor magnetic field. There is a back emf produced by the motor which is directly proportional to the angular velocity of the motor shaft with opposite polarity to the applied voltage $v_a$. The rotor has a moment of inertia $J$.

The field circuit voltage can be given by:

$$v_f = L_f \frac{di_f}{dt} + i_f R_f \tag{1}$$

The flux linkage in the armature winding due to the field winding can be expressed from the mutual inductance and the field current:

$$\psi_{ad} = L_{ad} i_f \tag{2}$$

The variation in flux linkage in the armature winding is a direct result of the armature rotation and thus the armature back emf can be expressed as:

$$e = \omega_r \psi_{ad} = \omega_r L_{ad} i_f \tag{3}$$

The armature equation during transient conditions is:

$$v_a = \omega_r L_{ad} i_f + L_a \frac{di_a}{dt} + i_a R_a \tag{4}$$

And the torque equation can be expressed as a function of $\psi_{ad}$ and thus $i_f$ as:

$$T = i_a \psi_{ad} = i_a L_{ad} i_f \tag{5}$$

# 4. Open Loop Control of a DC machine

Initially we will look at the open loop speed response of a DC machine before we 'close the loop' by applying feedback and a PID controller.

With the help of the ***'Simscape: Specialized Power Systems'*** library, we will construct a simple DC model in Simulink. On completion, the model should look like Fig. 5. Hint: do not forget the powergui block!

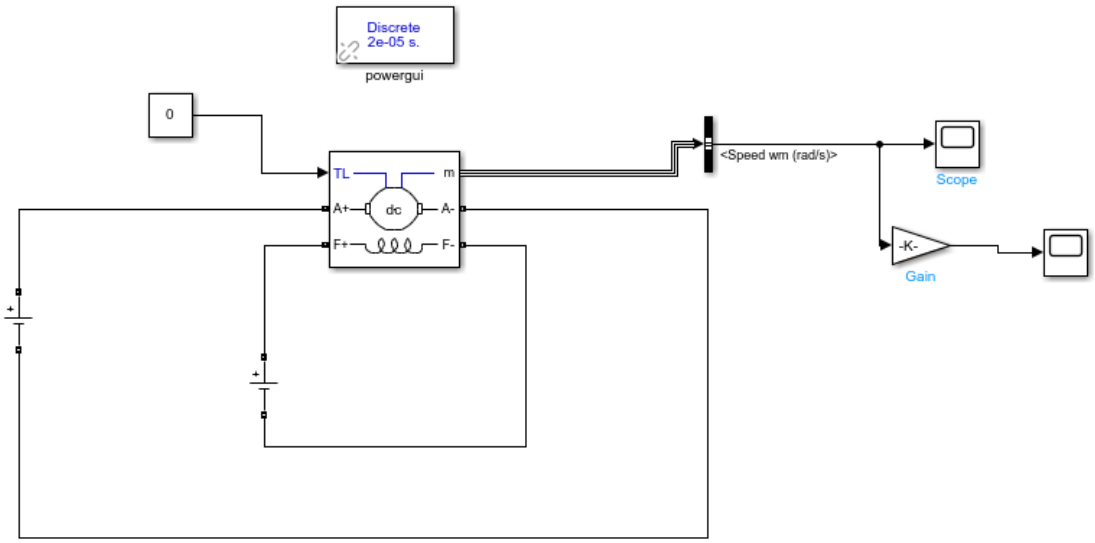


**Figure 5: Simple DC Machine Model**

The DC machine we are going to simulate is based on the ***'DC Machine'*** block. The machine model can be found in the ***'Simscape: Electrical: Specialized Power Systems: Electrical Machines'*** library. Choose ***'DC Machine'***. Set the machines ***'Configuration'*** as shown in Fig. 6 using the ***'Preset model'*** options, whilst in Fig. 7 make sure the ***'Initial speed'*** is set to zero. This will set the machine up as a wound field machine.

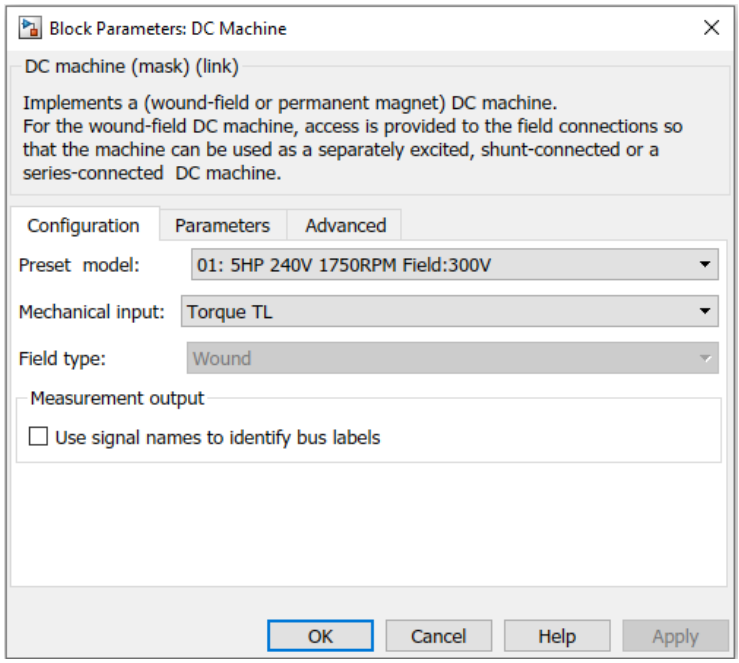


**Figure 6: DC Machine Configuration Parameters**

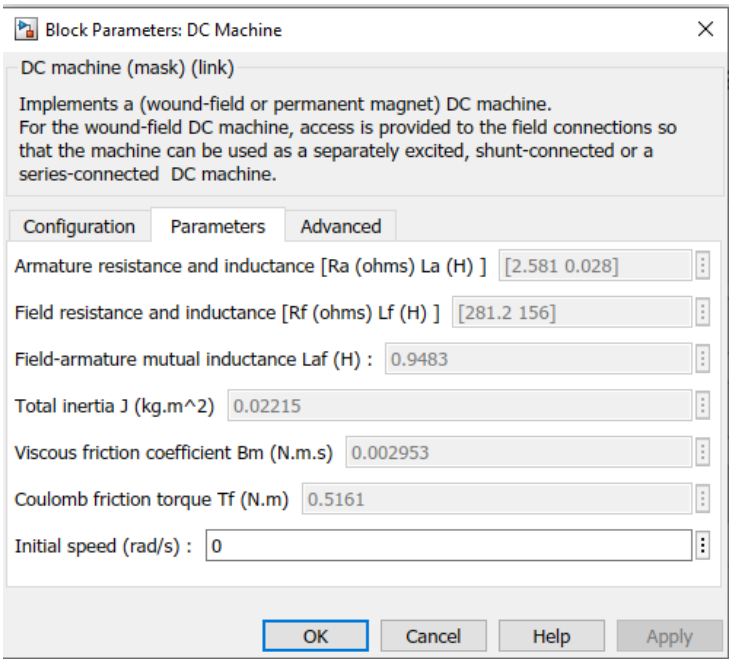


**Figure 7: DC Machine Configuration Parameters**

4 signals for the DC Machine can be obtained via the **'m'** port on the block; to choose which ones you want to view use the ***'Bus Selector'*** block, from ***'Simulink: Signal Routing'***. Double clicking on this block allows you to choose which signals you want to view. Initially we will just look at the motor speed (wm) in rad/s but have a look at the others for your own understanding.

You can also convert this speed in rad/s to the more usual rpm using a ***'Gain'*** block (***'Simulink: Commonly Used Blocks'***) and the following equation:

$$rpm = \frac{\omega_r}{2\pi} \times 60 \tag{6}$$

The ***'Tm'*** input on the ***'DC Machine'*** block allows you to apply load to the machine, a positive load means the machine is operating as a motor, a negative input means a generator. Here we will place a constant block from ***'Simulink: Commonly Used Blocks'***, initially set the value to 0 Nm.

You also need to add a ***'powergui'*** block, which can be found under ***'Simscape: Electrical: Specialized Power Systems'***. For the ***'powergui'*** settings, double click on the block and select the ***'Solver'*** options, choose ***'Simulation type: Discrete'*** and set the ***'Sample Time'*** to ***'1/50000'***; while in the ***'Preferences'*** options, set the ***'Discrete Solver'*** to ***'Tustin/Backward Euler (TBE)'***.

The ***'Scope'*** blocks can be found under ***'Simulink: Sinks'***. ***'Scope'*** blocks can have a limitation with the number of data points they save, it is recommended that you remove this limit. As shown in Fig. 8, if you click on the ***'Configuration Properties'*** button (highlighted in red) and click on the ***'Logging'*** tab, you can remove the limitation.

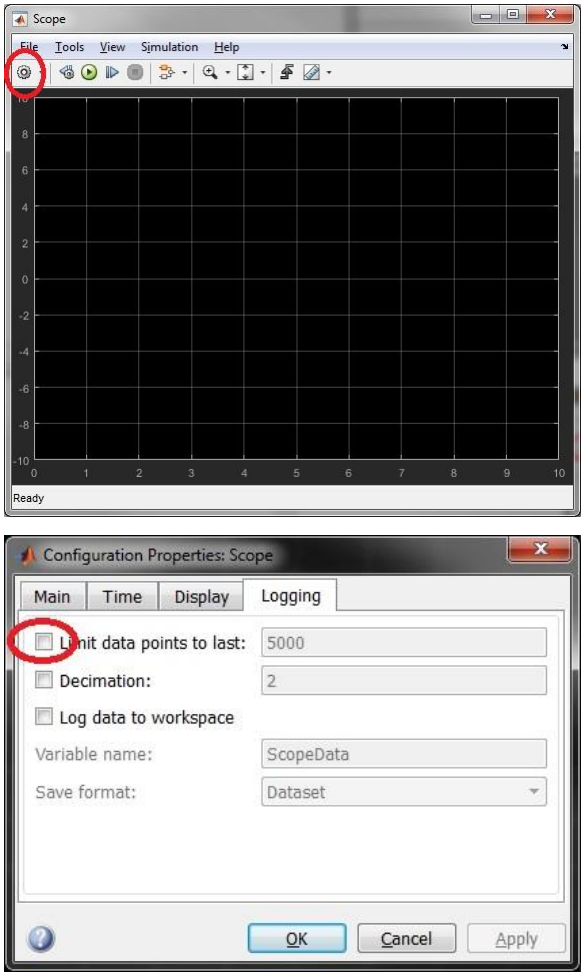


**Figure 8: Scope Parameters**

As the machine is a wound field machine, we need to apply voltages to both the field winding to generate a magnetic field in the machine, and the armature. For these, use the ***'DC Voltage Source'*** from ***'Simscape: Electrical: Specialized Power Systems: Sources'***. Double click on them to change the voltage. Set the field voltage to its rated 300 V, and set the armature voltage to 40 V.

## 4.1 Testing the System

If your simulation has been correctly implemented, it is now time to simulate the system. The simulation time and run buttons are highlighted in Fig. 9, but a more in-depth set of simulation settings can be accessed in the ***'Configuration Parameters'*** available from ***'Model Settings'*** under the ***'Modeling'*** tab. For this simulation set the stop time to 5 seconds and use the solver options as shown in Fig. 10.

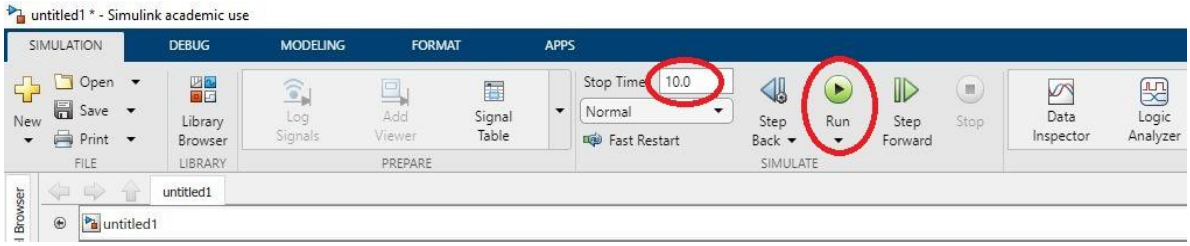


**Figure 9: Run and Time Buttons**

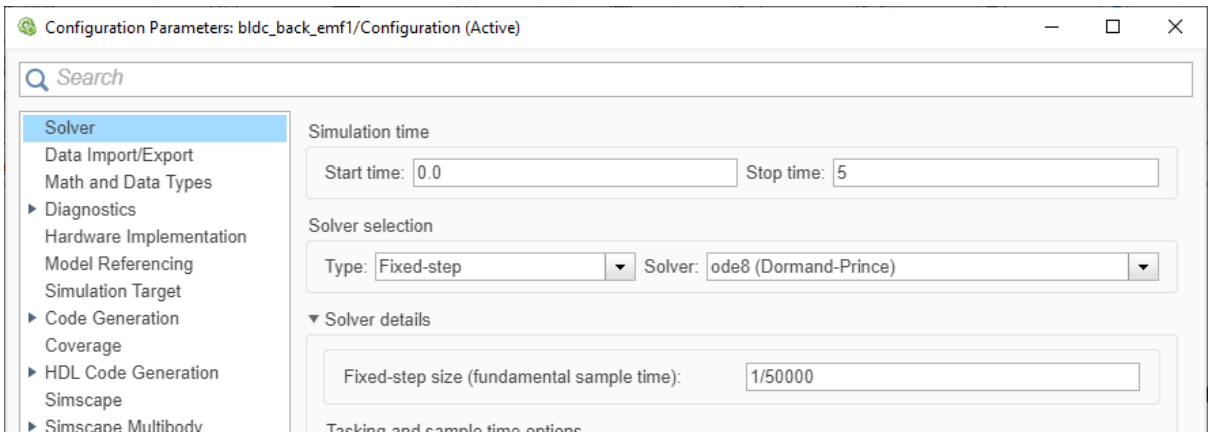


**Figure 10: Simulation Solver Settings**

Once the system had been built, run the simulation. The machine will accelerate to its rated speed condition which depends on the applied armature voltage. If your simulation is working correctly, the result for the rotor speed in rad/s should look like Fig. 11.

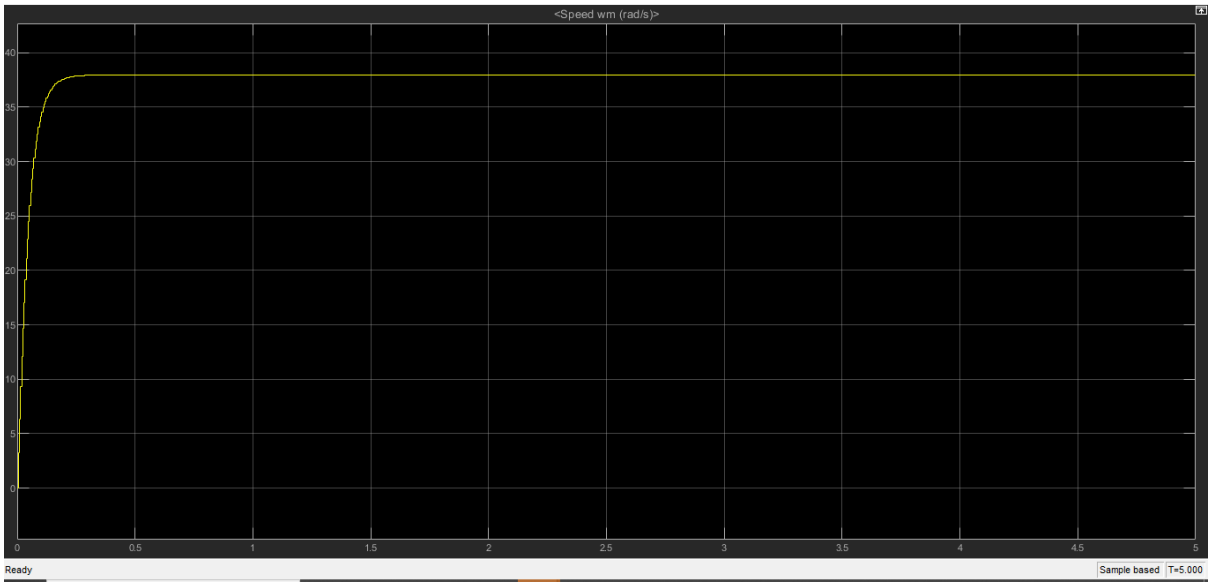


**Figure 11: Rotor Speed Response**

Run the simulation multiple times to get a steady state speed vs armature voltage profile:

| Armature Voltage (V) | Speed (rad/s) | Speed (rpm) |
|---|---|---|
| 0 | | |
| 40 | | |
| 80 | | |
| 120 | | |
| 160 | | |
| 200 | | |
| 240 | | |

What does this response look like?

In the systems configuration parameters, the speed of the machine was given as 1750 rpm at 240 V. Can you think why 2236 rpm is occurring for this voltage?

What torque is being applied to the machine?

We have set the constant value applied to ***'Tm'*** as 0 Nm, but the 1750 rpm will be the rated value, and this will be occurring when the rated power of 5 HP (horsepower) is occurring. To check this, we know that:

$$1 \text{ HP} = 745.7\ W \tag{7}$$

Therefore:

$$5 \text{ HP} = 5 \times 745.7 = 3728.5\ W \tag{8}$$

Using the standard power equation:

$$P = T\omega_r \tag{9}$$

The torque can be calculated from:

$$T = \frac{P}{\omega_r} \tag{10}$$

The rated rotor speed of 1750 rpm can be transformed to rad/s as this is the standard SI unit from:

$$\omega_r = \frac{rpm}{60} \times 2\pi \tag{11}$$

$$= \frac{1750}{60} \times 2\pi = 183.26\ rad/s \tag{12}$$

Therefore, the rated torque is:

$$T = \frac{3728.5}{183.26} = 20.35\ Nm \tag{13}$$

If you apply this load torque to the machine when the armature voltage is 240 V, what speed occurs?

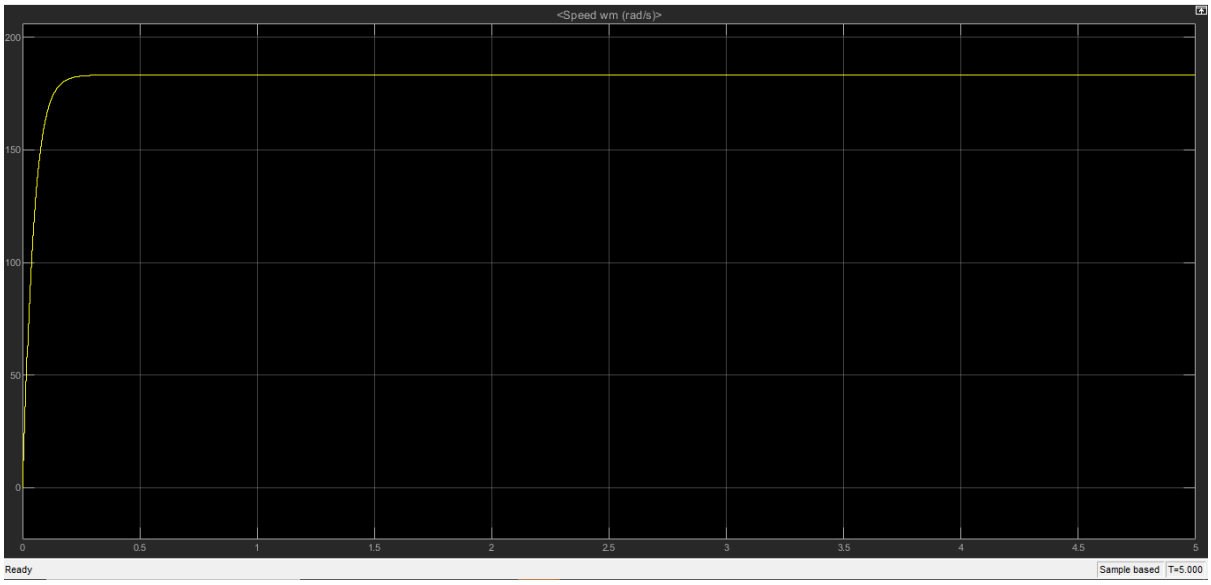


**Figure 12: Response at 20.35 Nm load**

Measuring this value gives 183.2 rad/s (1750 rpm), showing that the rated 1750 rpm occurs at the rated power.

Now find / calculate the voltage required for 1000 rpm to occur with no load applied. What is the voltage value?

If we now apply varying load to the system at this voltage. What occurs? Is the speed constant?

| Load (Nm) | Speed (rad/s) | Speed (rpm) |
|---|---|---|
| 0 | | |
| 4 | | |
| 8 | | |
| 12 | | |
| 16 | | |
| 20 | | |

You can obviously see that with load the system speed reduces. In most systems we cannot operate with open loop control as we need to have control of the speed, and often we need to keep it at a constant value irrespective of the load applied. To do this we need to apply feedback to the system to create a 'closed loop' system.

# 5. Overview of PID Control

PID (Proportional – Integral- Derivative) control is probably the most common method of closed loop control used worldwide, and approximately 90% of the control systems in the world use this class of controller due to its simplistic nature and effective robust responses.

As the name suggests, the control algorithm contains three basic coefficients, proportional, integral, and derivative, and the values of each of these can be varied to get the best system response.

The block diagram of a closed loop system is shown below:

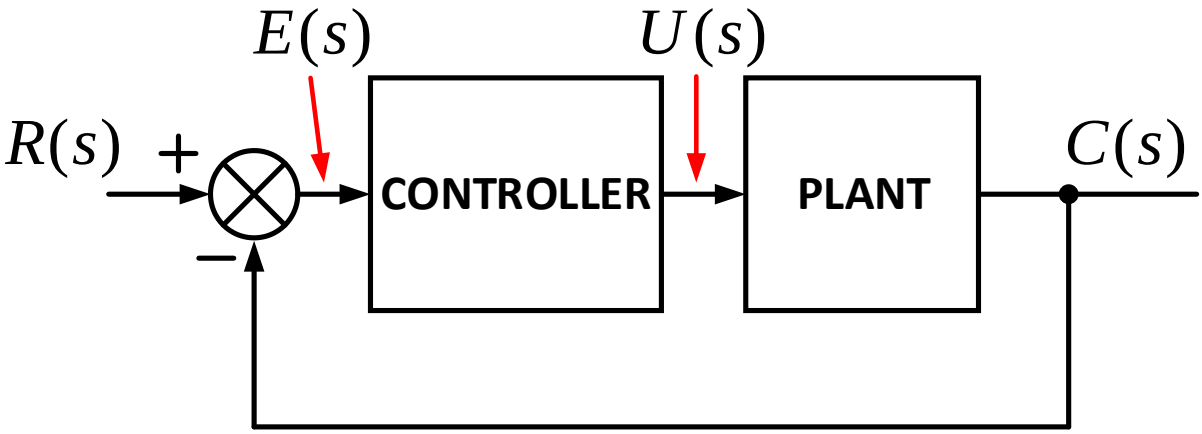


**Figure 13: Closed Loop System**

$R$ is the reference or desired value. $C$ is the output of the system. $E$ is the error signal applied to the controller (PID). $U$ is the control signal applied to the plant. For a PID controller this would be the proportional gain times the magnitude of the error, plus the integral gain times the integral of the error, plus the derivative gain times the derivative of the error.

The way this system works is that the control signal ($U$) is applied to the plant, a new output ($C$) is obtained, this new output is fed back and compared against the reference ($R$) which produces an error($E$). This new error is then applied to the controller which produces a new update to the control input, etc…

In the time domain a PID controller can be written as:

$$u(t) = K_p e(t) + K_i \int e(t)\, dt + K_d \frac{de(t)}{dt} \tag{14}$$

Or in the Laplace domain the PID controller becomes:

$$K_p + \frac{K_i}{s} + K_d s \tag{15}$$

But what do the different terms mean and do?

- The proportional component only depends on the difference between the reference and actual variables (error). The proportional gain ($K_p$) determines the ratio of the output response to the error signal. Generally, increasing the proportional gain will increase the speed of the control systems response. However, if the gain is too large, the system will begin to oscillate, and if the gain keeps increasing the system will become unstable. It does not always eliminate the steady state error and can lead to overshoot.
- The integral component sums the error term over time. This means that even a small error will cause the integral component to slowly increase, and the response will continually increase over time unless the steady state error is zero. However, it can make the system respond slower and oscillatory.
- The derivative component causes the output to decrease if the process variable is rapidly increasing, and the response is proportional to the rate of change of the error. Increasing the derivative value will increase the speed of the overall control system response, but most systems use very small value as it is highly sensitive to noise which can make the system unstable. The derivative term does not affect the steady state error.

You will learn more about PID control in your ENG2026 course.

The simple model of the closed loop DC motor PID system is shown in Fig. 14 below.

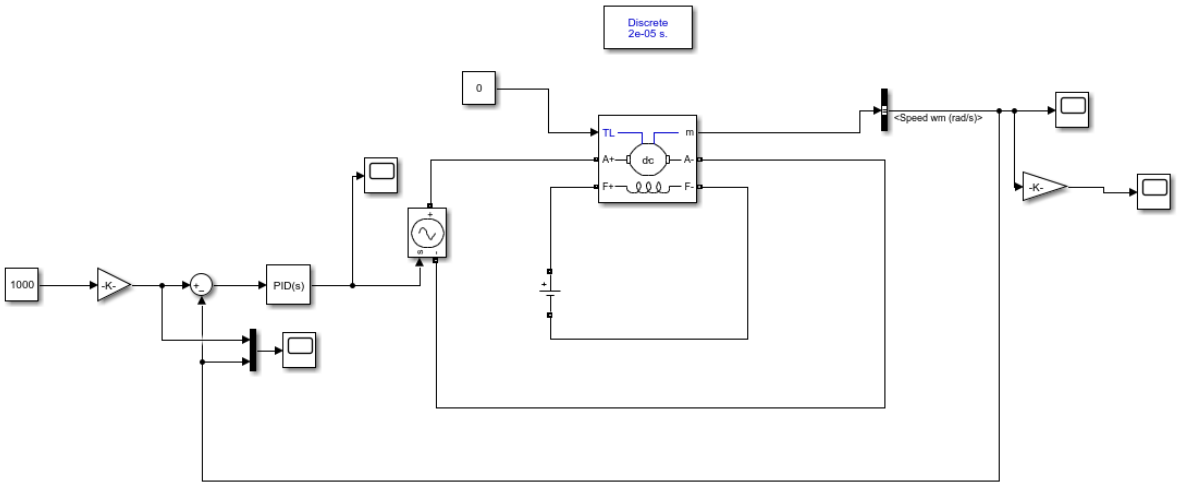


**Figure 14: Closed Loop DC Motor PID System**

In this system, the armature voltage DC source has been replaced by a ***'Controlled Voltage Source'*** obtained from the same library. This is controlled via a ***'PID Controller'*** from ***'Simulink: Continuous'***. The ***'Sum '*** block is from ***'Simulink: Commonly Used Blocks'***. Set the reference value to be 1000 rpm (104.8 rad/s) and convert this to the equivalent rad/s value to compare this to the measured signal.

In the PID controller set $K_p = 1$, and $K_d$ and $K_i$ to zero.
If you have set up your simulation correctly the response should look like this:

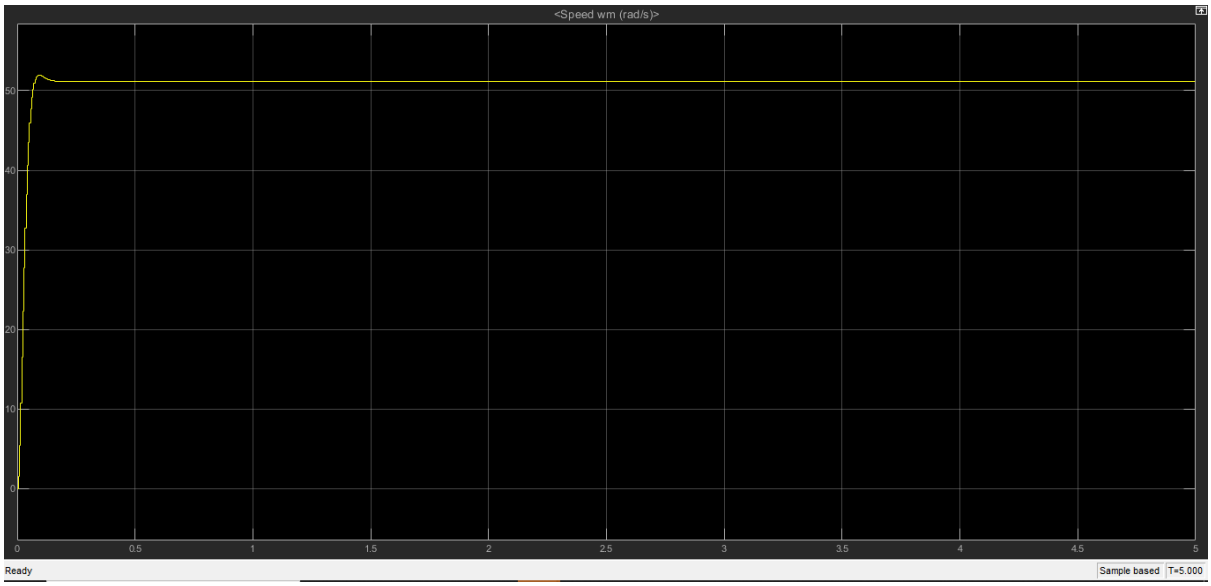


**Figure 15: $K_p = 1$, and $K_d$ and $K_i$ equal zero response**

You can see that there is a big steady state error.

Making $K_p = 10$ gives:

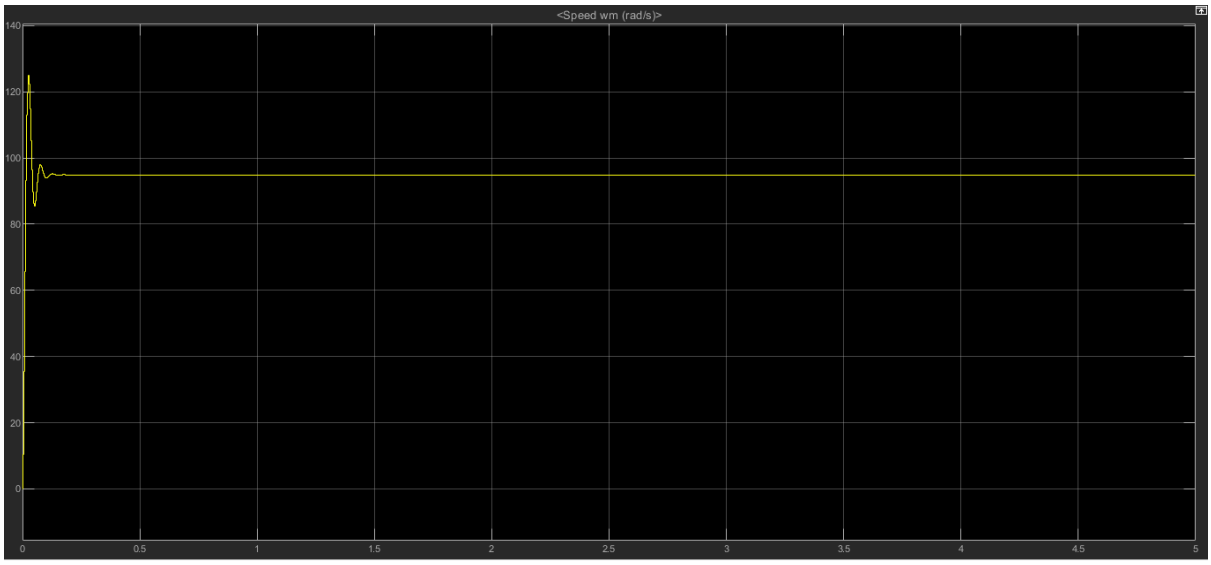


**Figure 16: $K_p = 10$, and $K_d$ and $K_i$ equal zero response**

Here you can see that the steady state error has reduced, but overshoot now occurs.

Making $K_p = 100$ gives:

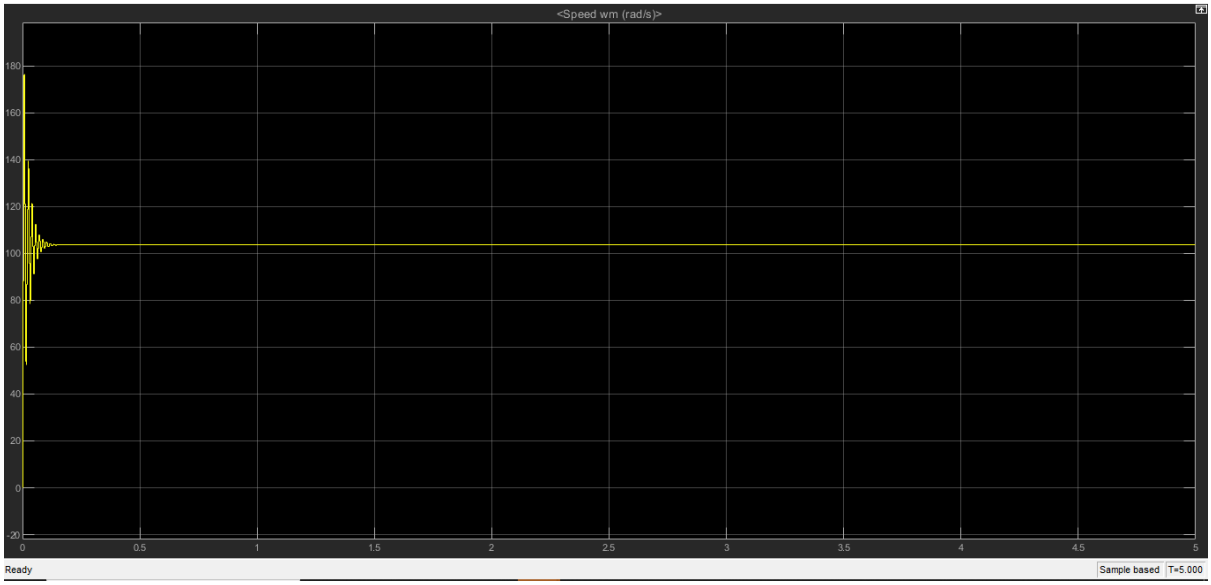


**Figure 17: $K_p = 100$, and $K_d$ and $K_i$ equal zero response**

Trying some other combinations looking at the effect of the integral and derivative terms.

In the PID controller set $K_d = 1$, and $K_p$ and $K_i$ to zero.

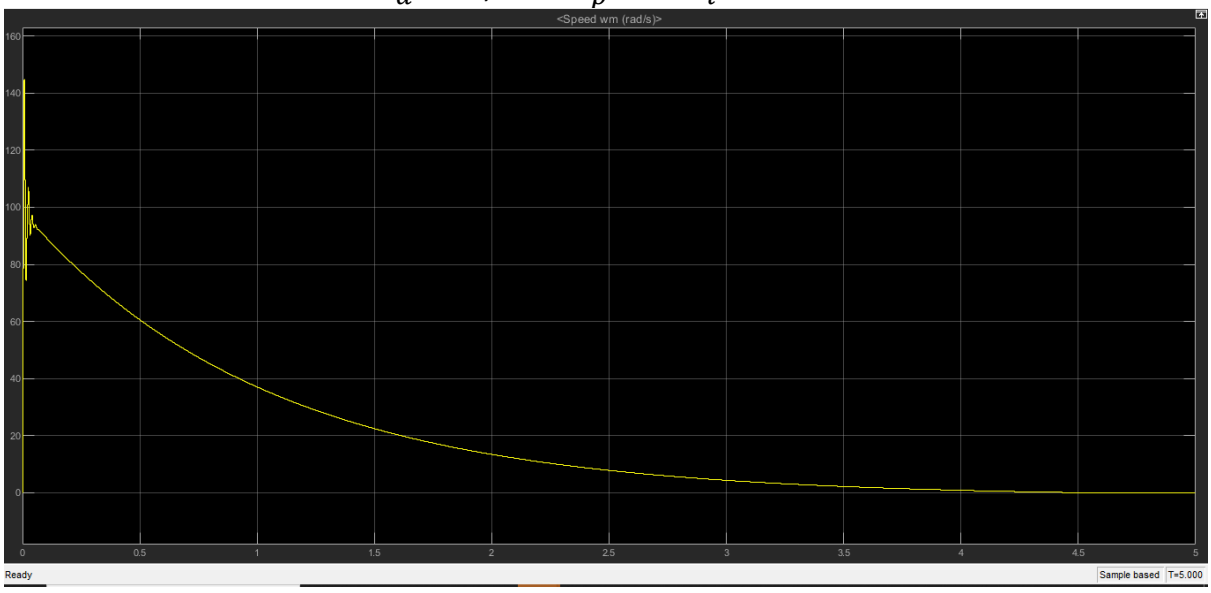


**Figure 18: $K_d = 1$, and $K_p$ and $K_i$ equal zero response.**

Making $K_d = 10$ gives:

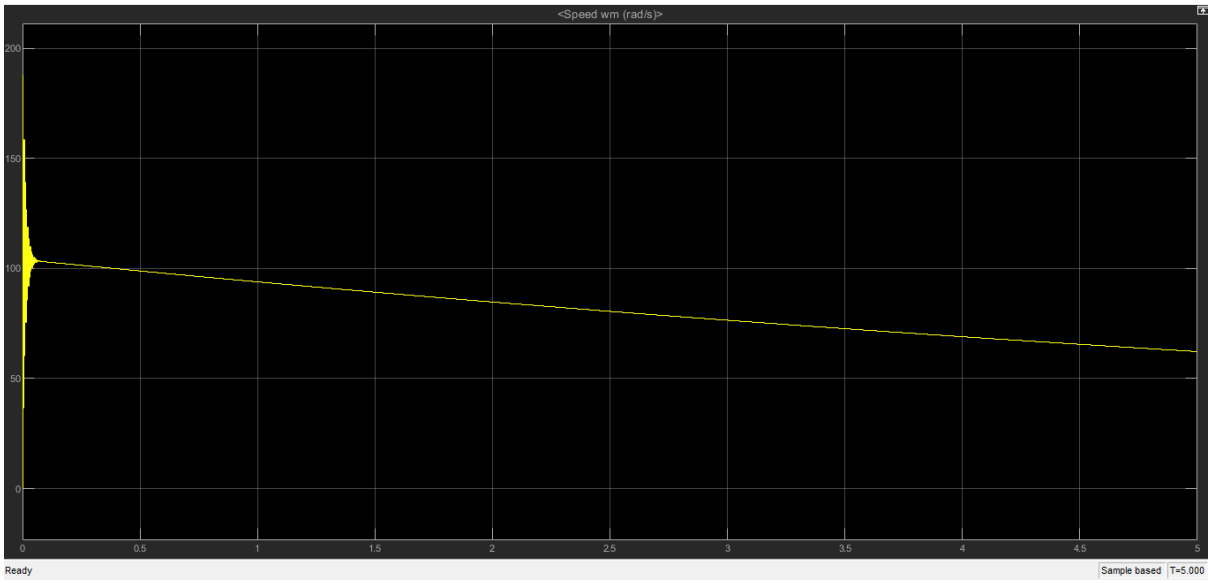


**Figure 19: $K_d = 10$, and $K_p$ and $K_i$ equal zero response.**

Making $K_d = 100$ gives:

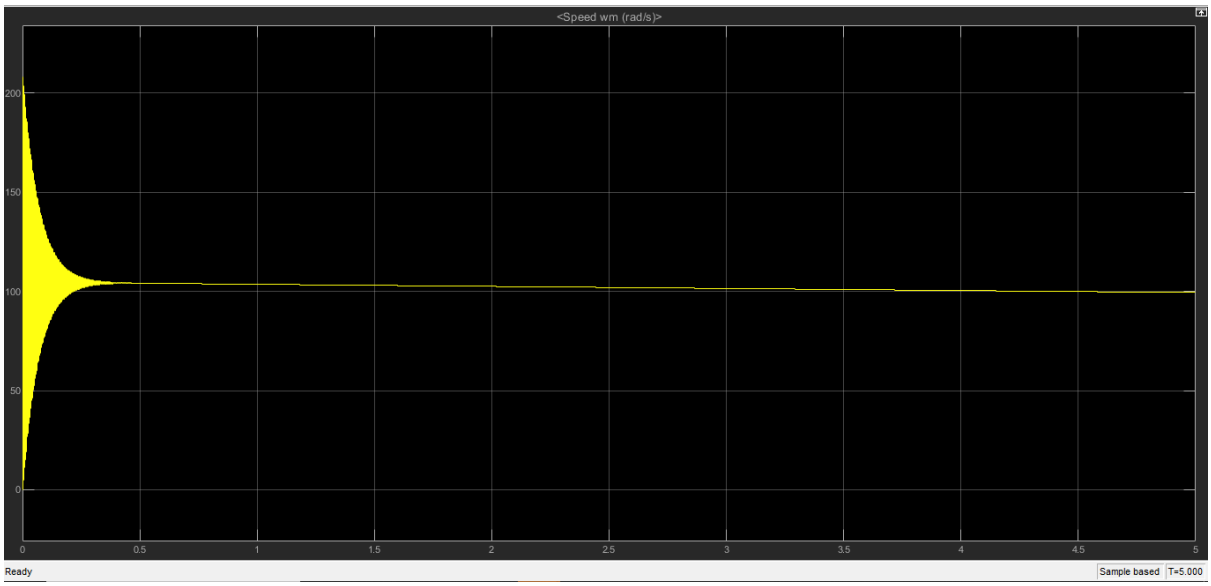


**Figure 20: $K_d = 100$, and $K_p$ and $K_i$ equal zero response.**

In the PID controller set $K_i = 1$, and $K_d$ and $K_p$ to zero.

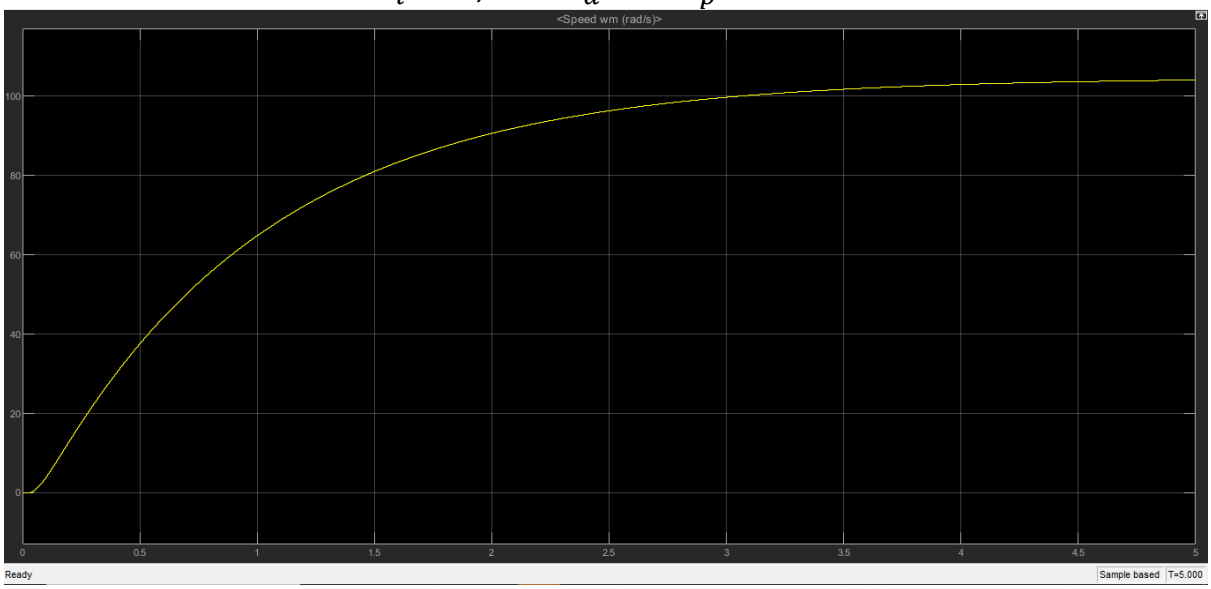


**Figure 21: $K_i = 1$, and $K_d$ and $K_p$ equal zero response**

Making $K_i = 10$ gives:

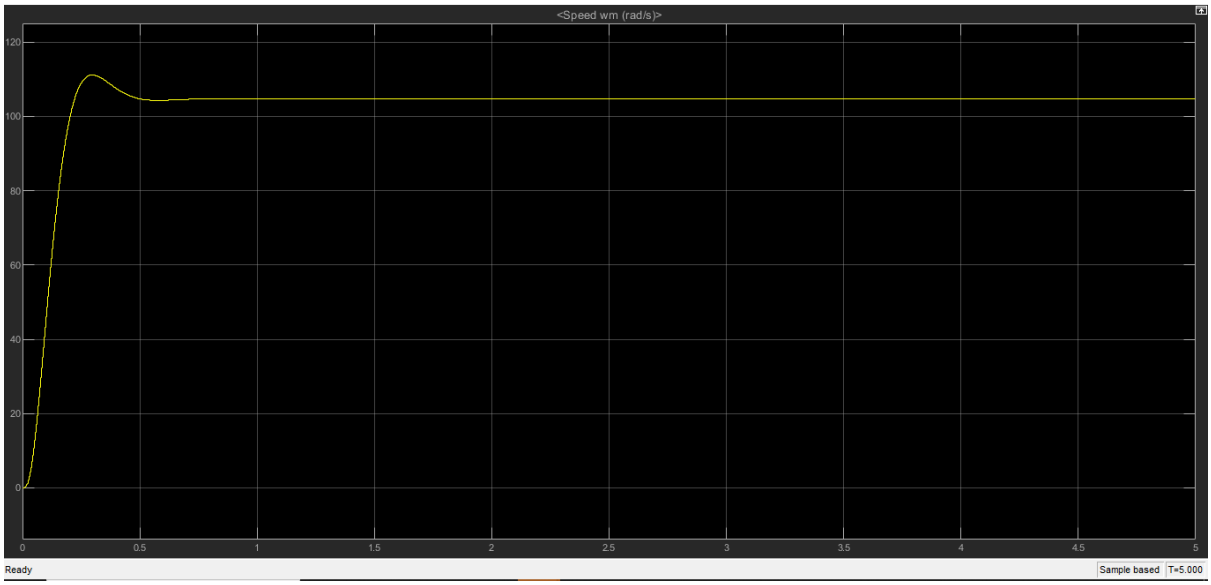


**Figure 22: $K_i = 10$, and $K_d$ and $K_p$ equal zero response**

Making $K_i = 100$ gives:

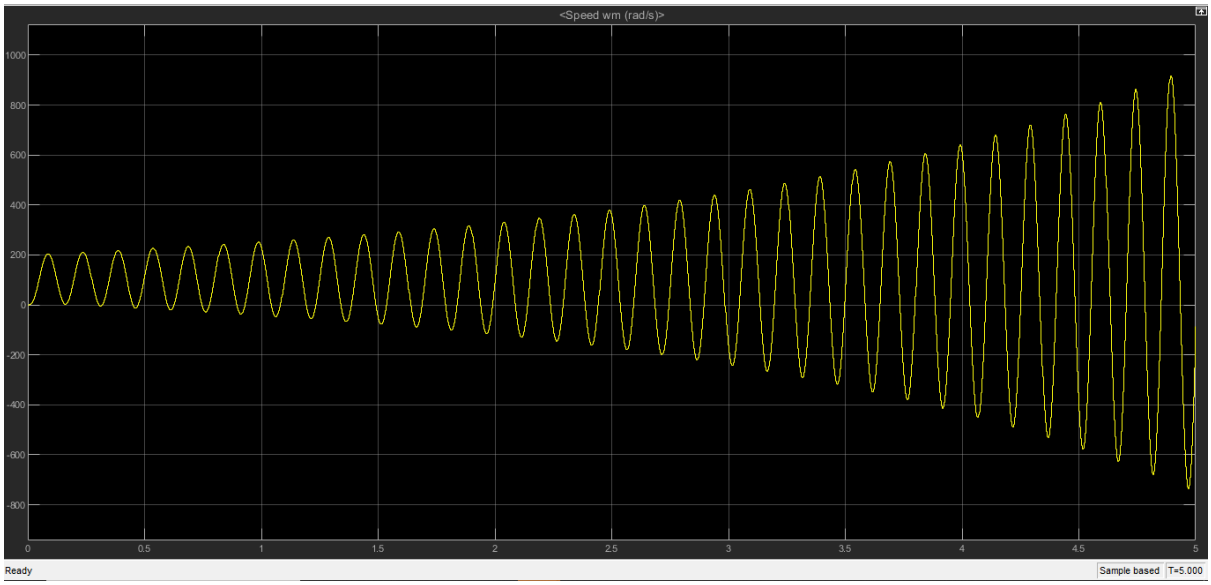


**Figure 23: $K_i = 100$, and $K_d$ and $K_p$ equal zero response**

Looking at these graphs you can see the effect that the different terms have on the speed signal.

Have a go and try and manually tune the $K_p$, $K_d$, and $K_i$ coefficients to get a sensible response with zero steady state.

What do you think of this manual "trial and error" method?

Once you have achieved a sensible speed response which tracks, apply some stepped load torque changes to see the effect. Does the speed return to the reference value? You may need to do some more tuning……

Another method for manually tuning PID controllers is shown below:

Manual method for tuning PID controllers.

1. Set $K_p$, $K_d$, and $K_i$ to zero.
2. Start with a small $K_p$, run the simulation, and keep doubling the value of $K_p$ until it starts to oscillate.
3. Half the value of $K_p$
4. Start with a small $K_d$, run the simulation, and keep doubling the value of $K_d$ until it starts to oscillate.
5. Half the value of $K_d$
6. Start with a small $K_i$, run the simulation, and keep doubling the value of $K_i$ until it starts to oscillate.
7. Half the value of $K_i$

If you are only using a PI controller, ignore the $K_d$ step, for a PD controller, ignore the $K_i$ step.

A more complicate tuning method is 'Ziegler Nichols' which looks at the oscillation period of the response, you can look into this if you wish.